

Лабораторная работа № 9

Детекция объектов с YOLO

Цель работы: научиться использовать современную архитектуру YOLO для детекции объектов на изображениях с помощью библиотеки Ultralytics.

Ход работы

1. Запустите поэтапно вспомогательный код, приведенный в работе. Разберитесь в работе кода.
2. Выполните детекцию на 3-5 своих фотографиях.
3. Измените порог уверенности и проанализируйте результаты.
4. Модифицируйте функцию `show_image_with_boxes`, добавив параметр `classes_to_show=None`, который позволяет показывать только определенные классы объектов (например, только людей или только машины).

```
def show_image_with_boxes_filtered(img, results, conf_threshold=0.5, classes_to_show=None):
```

```
# если classes_to_show = ['person'], показывать только людей
```

```
# если None - показывать все классы
```

Животные	Транспорт	Предметы в сумке/одежде	Еда	Мебель и интерьер
'cat' — кошка 'dog' — собака 'horse' — лошадь 'sheep' — овца 'cow' — корова 'elephant' — слон 'bear' — медведь 'zebra' — зебра 'giraffe' — жираф 'bird' — птица	'bicycle' — велосипед 'car' — автомобиль 'motorcycle' — мотоцикл 'airplane' — самолет 'bus' — автобус 'train' — поезд 'truck' — грузовик 'boat' — лодка 'traffic light' — светофор 'fire hydrant' — пожарный гидрант	'backpack' — рюкзак 'umbrella' — зонт 'handbag' — сумка 'tie' — галстук 'suitcase' — чемодан 'frisbee' — летающая тарелка 'skis' — лыжи 'snowboard' — сноуборд 'sports ball' — спортивный мяч	'banana' — банан 'apple' — яблоко 'sandwich' — бутерброд 'orange' — апельсин 'broccoli' — брокколи 'carrot' — морковь 'hot dog' — хот-дог 'pizza' — пицца 'donut' — пончик	'chair' — стул 'couch' — диван 'potted plant' — комнатное растение 'bed' — кровать 'dining table' — обеденный стол 'toilet' — унитаз 'tv' — телевизор 'laptop' — ноутбук 'mouse' — компьютерная мышь 'remote' — пульт 'keyboard' — клавиатура 'cell phone' — сотовый телефон
Дорожные знаки и разметка	Спортивный инвентарь	Кухонная утварь		
'stop sign' — знак "стоп" 'parking meter' — паркомат 'bench' — скамейка	'kite' — воздушный змей 'baseball bat' — бейсбольная бита 'baseball glove' — бейсбольная перчатка 'skateboard' — скейтборд 'surfboard' — серфборд 'tennis racket' — теннисная ракетка	'bottle' — бутылка 'wine glass' — бокал для вина 'cup' — чашка 'fork' — вилка 'knife' — нож 'spoon' — ложка 'bowl' — миска		
Человек	Бытовая техника			
'person' — человек	'microwave' — микроволновка 'oven' — духовка 'toaster' — тостер 'sink' — раковина 'refrigerator' — холодильник			

5. Напишите функцию `process_video(video_path, output_path)`, которая обрабатывает видеофайл и сохраняет результат с детекцией на каждом кадре (используйте вспомогательный код).

1. Этап. Установка библиотек:

```
!pip install ultralytics opencv-python matplotlib numpy
```

```
import cv2
import torch
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

```
from PIL import Image
import requests
from io import BytesIO
```

Пояснение

ultralytics – главная библиотека для YOLO (загрузка предобученных моделей YOLO, детекция объектов на изображениях и видео, обучение своих моделей, экспорт моделей в разные форматы)

opencv – работа с изображениями.

Взаимодействие в коде:

```
# 1. Скачиваем изображение из интернета
response = requests.get(url) # requests
# 2. Преобразуем в изображение
img = Image.open(BytesIO(response.content)) # PIL + BytesIO
# 3. Конвертируем в NumPy массив
img = np.array(img) # numpy
# 4. Загружаем YOLO модель
model = YOLO('yolov8n.pt') # ultralytics
# 5. Выполняем детекцию (PyTorch внутри)
results = model(img) # ultralytics + torch
# 6. Рисуем прямоугольники (OpenCV)
for box in boxes:
    cv2.rectangle(img_copy, (x1, y1), (x2, y2), (0,255,0), 2) # opencv
# 7. Показываем результат (Matplotlib)
plt.imshow(img_copy) # matplotlib
plt.show()
```

2.Этап. Загрузка предобученной модели YOLO

```
model = YOLO('yolov8n.pt') # nano - самая маленькая и быстрая модель
print("Модель YOLOv8 успешно загружена!")
print(f"Доступные классы: {model.names}")
print(f"Всего классов: {len(model.names)}")
```

3.Этап. Функции для загрузки и отображения изображений

```
def load_image_from_url(url):
    """Загрузка изображения по URL"""
    response = requests.get(url)
    img = Image.open(BytesIO(response.content))
    return np.array(img)

def load_image_from_file(path):
    """Загрузка изображения из файла"""
    img = cv2.imread(path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    return img

def show_image_with_boxes(img, results, conf_threshold=0.5):
    """
    Отображение изображения с детектированными объектами
    """
    if isinstance(results, list):
        result = results[0]
    else:
```

```

result = results

# Получаем боксы, классы и уверенность
boxes = result.bboxes
img_copy = img.copy()

if boxes is not None:
    for box in boxes:
        # Получаем координаты бокса
        x1, y1, x2, y2 = box.xyxy[0].cpu().numpy().astype(int)
        conf = box.conf[0].cpu().numpy()
        cls = int(box.cls[0].cpu().numpy())

        # Фильтруем по уверенности
        if conf < conf_threshold:
            continue

        # Рисуем прямоугольник
        cv2.rectangle(img_copy, (x1, y1), (x2, y2), (0, 255, 0), 2)

        # Добавляем подпись
        label = f"{model.names[cls]}: {conf:.2f}"
        cv2.putText(img_copy, label, (x1, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

# Отображаем результат
plt.figure(figsize=(12, 8))
plt.imshow(img_copy)
plt.axis('off')
plt.title(f'Детекция объектов (порог уверенности: {conf_threshold})')
plt.show()

return img_copy

```

Пояснение

Функция загрузки изображения по URL. Шаги:

1. requests.get(url) – как будто зашли в браузер по ссылке и скачали картинку. В response теперь лежат байты изображения.
2. BytesIO(response.content) – создает "виртуальный файл" в оперативной памяти из скачанных байтов. Это нужно, чтобы Image.open() могла прочитать данные как файл, но без сохранения на диск.
3. Image.open(...) – открывает изображение из виртуального файла с помощью PIL.
4. np.array(img) – преобразует PIL-изображение в массив NumPy. Теперь с ним можно работать через OpenCV, Matplotlib и YOLO.

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # 2. Меняем порядок цветов

OpenCV читает изображения в формате BGR, Matplotlib и YOLO ожидают формат RGB.

conf_threshold – порог уверенности.

result.bboxes содержит все найденные объекты. Каждый объект имеет: координаты рамки, класс объекта, уверенность.

4.Этап. Детекция на тестовых изображениях

```

# Пример 1: Детекция на изображении из интернета (уличная сцена)
print("Пример 1: Детекция на уличной сцене")

```

```

url =
"https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/
bus.jpg"
img1 = load_image_from_url(url)

# Выполняем детекцию
results1 = model(img1)

# Показываем результат
show_image_with_boxes(img1, results1, conf_threshold=0.5)

# Пример 2: Детекция на другом изображении
print("Пример 2: Детекция на изображении с животными")
url2 =
"https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/
zidane.jpg"
img2 = load_image_from_url(url2)

# Выполняем детекцию
results2 = model(img2)

# Показываем результат
show_image_with_boxes(img2, results2, conf_threshold=0.5)

```

5.Этап. Анализ результатов детекции

```

def analyze_detection_results(results, img_name):
    """
    Анализ результатов детекции
    """
    if isinstance(results, list):
        result = results[0]
    else:
        result = results

    boxes = result.boxes

    if boxes is None:
        print("Объекты не обнаружены")
        return

    # Собираем статистику
    detections = []
    for box in boxes:
        conf = float(box.conf[0].cpu().numpy())
        cls = int(box.cls[0].cpu().numpy())
        class_name = model.names[cls]
        detections.append({
            'class': class_name,
            'confidence': conf
        })

    # Выводим статистику
    print(f"\nРезультаты детекции для {img_name}:")
    print(f"Всего обнаружено объектов: {len(detections)}")
    print(f"\nОбнаруженные объекты:")

    # Группируем по классам
    from collections import Counter
    class_counts = Counter([d['class'] for d in detections])
    for class_name, count in class_counts.items():
        print(f"  {class_name}: {count}")

    print(f"\nДетальная информация:")
    for i, det in enumerate(detections, 1):

```

```

        print(f"    {i}. {det['class']} (уверенность: {det['confidence']:.3f})")

    return detections

# Анализируем результаты для первого изображения
detections1 = analyze_detection_results(results1, "уличная сцена")

```

6. Этап. Детекция с разными порогами уверенности

```

def compare_confidence_thresholds(img, thresholds=[0.3, 0.5, 0.7, 0.9]):
    """
    Сравнение детекции с разными порогами уверенности
    """
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))
    axes = axes.ravel()

    for i, conf_thresh in enumerate(thresholds):
        # Выполняем детекцию
        results = model(img)

        if isinstance(results, list):
            result = results[0]
        else:
            result = results

        boxes = result.bboxes
        img_copy = img.copy()

        if boxes is not None:
            for box in boxes:
                x1, y1, x2, y2 = box.xyxy[0].cpu().numpy().astype(int)
                conf = box.conf[0].cpu().numpy()
                cls = int(box.cls[0].cpu().numpy())

                # Фильтруем по порогу
                if conf >= conf_thresh:
                    cv2.rectangle(img_copy, (x1, y1), (x2, y2), (0, 255, 0), 2)
                    label = f"{model.names[cls]}: {conf:.2f}"
                    cv2.putText(img_copy, label, (x1, y1-10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

            axes[i].imshow(img_copy)
            axes[i].set_title(f'Порог уверенности: {conf_thresh}')
            axes[i].axis('off')

    plt.tight_layout()
    plt.show()

# Сравниваем пороги на первом изображении
compare_confidence_thresholds(img1)

```

7. Этап. Детекция на собственном изображении. Введите название собственного изображения или путь к нему

```

def detect_on_my_image(image_path):
    """
    Детекция на пользовательском изображении
    """
    try:
        # Загружаем изображение
        if image_path.startswith('http'): # если это URL
            img = load_image_from_url(image_path)
        else: # если это локальный файл
            img = load_image_from_file(image_path)

```

```

print(f"Изображение загружено. Размер: {img.shape}")

# Выполняем детекцию
results = model(img)

# Показываем результат
result_img = show_image_with_boxes(img, results)

# Анализируем результаты
analyze_detection_results(results, "пользовательское изображение")

return results

except Exception as e:
    print(f"Ошибка при обработке изображения: {e}")
    return None

# здесь нужно указать имя вашего файла:
results = detect_on_my_image("улица.png")
# или путь
# results = detect_on_my_image("/home/user/Pictures/моя_картинка.jpg")

```

8. Этап. Сохранение результатов

```

def save_detection_results(img, results, output_path="detection_result.jpg"):
    """
    Сохранение изображения с результатами детекции
    """
    if isinstance(results, list):
        result = results[0]
    else:
        result = results

    # Получаем изображение с нарисованными боксами
    annotated_img = result.plot()

    # Конвертируем BGR в RGB для корректного сохранения
    annotated_img_rgb = cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB)

    # Сохраняем
    cv2.imwrite(output_path, cv2.cvtColor(annotated_img_rgb, cv2.COLOR_RGB2BGR))
    print(f"Результат сохранен в {output_path}")

    return annotated_img_rgb

# Сохраняем результат для первого изображения
saved_img = save_detection_results(img1, results1, "yolo_detection_bus.jpg")

```

9. Этап. Интерактивная демонстрация

```

def interactive_detection_fixed():
    """
    Исправленная версия с автоматическим закрытием окон
    """
    print("=" * 50)
    print("ДЕМОНСТРАЦИЯ YOLO ДЕТЕКЦИИ")
    print("=" * 50)

    # Список доступных тестовых изображений
    test_images = {
        "1": "Уличная сцена (автобус, люди)",
        "2": "Спортивная сцена (футболисты)",
        "3": "Своё изображение (URL или путь)"
    }

```

```

}

print("\nДоступные изображения для теста:")
for key, desc in test_images.items():
    print(f"    {key}. {desc}")

choice = input("\nВыберите вариант (1-3): ")

if choice == "1":
    url =
"https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/
bus.jpg"
    img = load_image_from_url(url)
    img_name = "уличная сцена"
elif choice == "2":
    url =
"https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/
zidane.jpg"
    img = load_image_from_url(url)
    img_name = "спортивная сцена"
elif choice == "3":
    path = input("Введите URL или путь к изображению: ")
    img = detect_on_my_image(path)
    if img is None:
        return
    img_name = "пользовательское изображение"
else:
    print("Неверный выбор")
    return

print(f"\nОбработка {img_name}...")
results = model(img)

print("\nРезультаты детекции:")
analyze_detection_results(results, img_name)

# Показываем изображение и автоматически закрываем через 5 секунд
print("\nОтображаем результат... Окно закроется через 5 секунд")
show_image_with_boxes(img, results)

# Таймер для автоматического закрытия
import time
time.sleep(5)
plt.close('all')

# Спрашиваем, сохранить ли результат
save_choice = input("\nСохранить результат? (y/n): ")
if save_choice.lower() == 'y':
    filename = input("Имя файла для сохранения (по умолчанию: yolo_result.jpg): ")
    if not filename:
        filename = "yolo_result.jpg"
    save_detection_results(img, results, filename)

print("Программа завершена!")

# Запуск
interactive_detection_fixed()

```

10.Этап. Детекция с разными моделями YOLO

```

def compare_yolo_models(image_path):
    """
    Сравнение разных версий YOLO моделей
    """

```

```

img = load_image_from_url(image_path) if image_path.startswith('http') else
load_image_from_file(image_path)

# Список моделей для сравнения
models_to_try = {
    'YOLOv8n (nano)': 'yolov8n.pt',
    'YOLOv8s (small)': 'yolov8s.pt',
    'YOLOv8m (medium)': 'yolov8m.pt'
}

fig, axes = plt.subplots(1, 3, figsize=(20, 8))

for idx, (model_name, model_path) in enumerate(models_to_try.items()):
    print(f"Загрузка {model_name}...")

    # Загружаем модель (скачается автоматически)
    current_model = YOLO(model_path)

    # Выполняем детекцию
    results = current_model(img)

    # Получаем результат с боксами
    result_img = results[0].plot()
    result_img_rgb = cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB)

    # Отображаем
    axes[idx].imshow(result_img_rgb)
    axes[idx].set_title(f'{model_name}')
    axes[idx].axis('off')

    # Считаем количество объектов
    num_objects = len(results[0].boxes) if results[0].boxes else 0
    print(f" Обнаружено объектов: {num_objects}")

plt.tight_layout()
plt.show()

# Сравниваем модели на тестовом изображении
compare_yolo_models("https://raw.githubusercontent.com/ultralytics/ultralytics/main
/ultralytics/assets/bus.jpg")

```

Обработка видео

```

def detect_on_webcam():
    """
    Детекция объектов с веб-камеры в реальном времени
    Нажмите 'q' для выхода
    """
    # Запускаем веб-камеру
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Ошибка: Не удалось открыть веб-камеру")
        return

    print("Детекция в реальном времени. Нажмите 'q' для выхода...")

    while True:
        # Захват кадра
        ret, frame = cap.read()

```



```

        if not ret:
            break

        # Детекция на кадре
        results = model(frame, verbose=False) # verbose=False отключает вывод в
        консоль

        # Отрисовка результатов
        annotated_frame = results[0].plot()

        # Отображение
        cv2.imshow('YOLOv8 Детекция', annotated_frame)

        # Выход по нажатию 'q'
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

# Раскомментируйте для запуска с веб-камерой
# detect_on_webcam()

# ===== обработка видеофайла =====
def process_video(video_path, output_path, conf_threshold=0.5):
    """
    Обработка видеофайла с детекцией объектов на каждом кадре

    Параметры:
    video_path - путь к входному видеофайлу
    output_path - путь для сохранения обработанного видео
    conf_threshold - порог уверенности (по умолчанию 0.5)
    """
    # Открываем видеофайл
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print(f"Ошибка: Не удалось открыть видеофайл {video_path}")
        return

    # Получаем параметры видео
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    print(f"Видео загружено: {video_path}")
    print(f"Размер: {width}x{height}, FPS: {fps}, Всего кадров: {total_frames}")

    # Создаем объект для записи видео
    fourcc = cv2.VideoWriter_fourcc(*'mp4v') # кодек для MP4
    out = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

    frame_count = 0

```

```

print("Обработка видео...")

while True:
    # Читаем кадр
    ret, frame = cap.read()
    if not ret:
        break

    # Выполняем детекцию на кадре
    results = model(frame, conf=conf_threshold, verbose=False)

    # Получаем кадр с нарисованными боксами
    annotated_frame = results[0].plot()

    # Записываем обработанный кадр
    out.write(annotated_frame)

    # Обновляем счетчик и выводим прогресс
    frame_count += 1
    if frame_count % 30 == 0: # показываем прогресс каждые 30 кадров
        print(f"Обработано кадров: {frame_count}/{total_frames}
({frame_count/total_frames*100:.1f}%)")

    # Освобождаем ресурсы
    cap.release()
    out.release()
    cv2.destroyAllWindows()

print(f"\nОбработка завершена!")
print(f"Обработано кадров: {frame_count}")
print(f"Результат сохранен в: {output_path}")
# ВЫЗОВ ФУНКЦИИ - раскомментируйте и укажите пути к вашему видео
video_input = "1k.mp4" # замените на путь к вашему видеофайлу
video_output = "результат_видео.mp4" # имя выходного файла

process_video(video_input, video_output, conf_threshold=0.5)

```